

Entrega 1

Proyecto

final

Análisis y diseño de algoritmos

Indice

Definición del problema:	2
Entradas:	2
Salidas:	3
Restricciones:	3
Solución Fuerza bruta	4
Análisis de complejidad — Fuerza Bruta:	4
Gráfica comparativa (tiempo vs tamaño de entrada):	5
Solución recursiva	6
Análisis de complejidad	6
Análisis:	7
Conclusión:	7

Definición del problema:

Este proyecto aborda un problema sobre un sistema de logística y entregas de una empresa de paquetería. La empresa cuenta con una flota de camiones encargados de transportar paquetes hacia diferentes destinos, teniendo en cuenta restricciones de capacidad y tiempos de entrega.

El problema consiste en determinar y asignar eficientemente paquetes a camiones, buscando maximizar la cantidad de entregas realizadas sin exceder la capacidad de carga de los vehículos y respetando las restricciones temporales de cada paquete.

Este tipo de problema es común en sistemas reales de distribución, comercio electrónico y cadenas de suministro, donde una mala planificación puede generar retrasos, sobrecostos y baja eficiencia operativa.

Entradas:

Paquetes (n paquetes)

Con la siguiente información:

Campo	Descripción
ID	Identificador único del paquete
Peso	Peso del paquete
Destino	Lugar de entrega
Tiempo límite	Tiempo máximo permitido para la entrega
Prioridad	Nivel de prioridad del paquete

Ejemplo: 1, 10, A, 5, 1

Representa:

- Paquete 1
- Peso: 10
- Destino: A
- Tiempo límite: 5
- Prioridad: 1

Camiones (m camiones)

Campo	Descripcion
ID	Identificador del camión
Capacidad	Peso máximo soportado

Ejemplo: 101, 40

Representa:

- Camión 101
- Capacidad máxima: 40

Salidas:

La salida esperada del sistema es una asignación de paquetes a camiones que indique:

- Qué paquetes serán transportados por cada camión
- Que ninguna capacidad sea excedida
- El conjunto de entregas realizadas

Restricciones:

El problema debe cumplir las siguientes restricciones:

1. La suma de pesos de los paquetes asignados a un camión no puede superar su capacidad máxima.
2. Cada paquete puede asignarse como máximo a un solo camión.
3. Deben considerarse las restricciones temporales de entrega.
4. Se busca maximizar la eficiencia de distribución y la cantidad de paquetes entregados.

Solución Fuerza bruta

Análisis de complejidad — Fuerza Bruta:

Complejidad temporal: $O((m + 1)^n)$

La solución explora todas las formas posibles de asignar n paquetes a m camiones. Para cada paquete hay $m + 1$ opciones (los m camiones más la opción de no asignarlo), y esto se repite para los n paquetes, lo que genera $(m + 1)^n$ combinaciones en total.

Por ejemplo:

- Con 5 paquetes y 2 camiones se exploran $3^5 = 243$ operaciones.
- Con 15 paquetes y 4 camiones se exploran $5^{15} = 30.517.578.125$ operaciones.
- Con 40 paquetes y 8 camiones se exploran 9^{40} operaciones, un número tan grande que es computacionalmente inviable, es por eso por lo que el sistema no arroja ningún resultado.

Estos son ejemplos arbitrarios, ya que los que construimos nosotros incluyen una mayor cantidad de valores, lo que permitirá que en algoritmos posteriores se observe con mayor claridad la variación temporal entre ellos.

Complejidad espacial:

$O(n * m)$

En memoria se almacenan la asignación actual (una lista por camión con los paquetes asignados) y las cargas actuales de cada camión. El espacio máximo usado en cualquier momento es proporcional al número de paquetes por el número de camiones.

Justificación paso a paso:

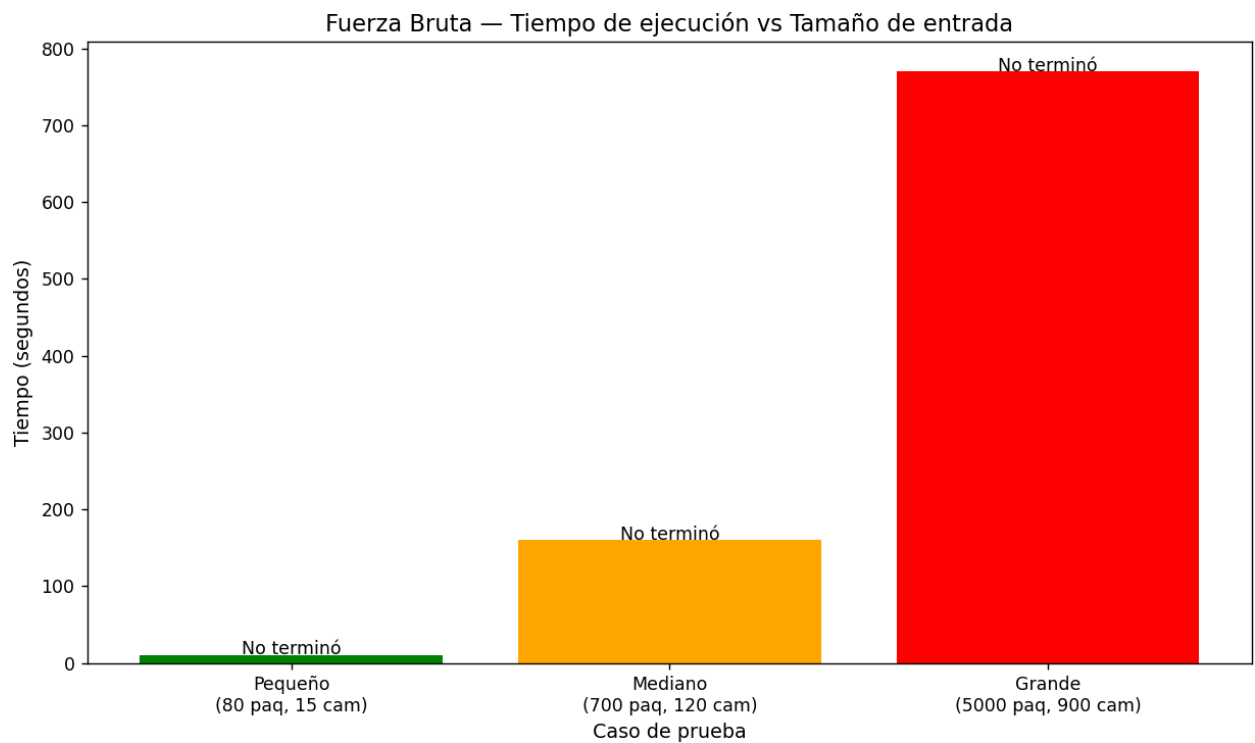
1. $product(range(m + 1), repeat = n)$ genera $(m + 1)^n$ combinaciones en total
2. Por cada combinación se itera sobre los n paquetes para validar capacidad: $O(n)$
3. Por cada combinación válida se recorren los m camiones para guardar el resultado: $O(m)$
4. Total por combinación: $O(n + m)$

Medición empírica de tiempos con diferentes tamaños de entrada:

Caso	Paquetes	Camiones	Combinaciones $(m+1)^n$	Tiempo real
Pequeño	80	15	$16^{80} \approx \dots$	No termina
Mediano	700	120	$121^{700} \approx \dots$	No termina
Grande	5000	900	$901^{5000} \approx \dots$	No termina

La fuerza bruta con nuestros ejemplos no es viable. A medida que aumentan los paquetes y camiones el tiempo crece de forma exponencial, haciendo imposible su uso en casos reales. Así se justifica la necesidad de usar algoritmos más eficientes como greedy, backtracking y divide y vencerás para los casos propuestos, y esto lo veremos más adelante.

Gráfica comparativa (tiempo vs tamaño de entrada):



Solución recursiva

Implementación

La solución recursiva aborda el problema de asignación de paquetes procesando cada paquete uno por uno mediante llamadas recursivas. A diferencia de la fuerza bruta que explora todas las combinaciones posibles, esta solución toma la primera decisión válida que encuentra y continúa sin reconsiderar.

Identificación del caso base y caso recursivo

Caso base:

Cuando **índice == len(paquetes)** — ya no hay más paquetes que procesar, se retorna la asignación actual tal como está.

Caso recursivo:

Para cada paquete en la posición índice:

- Recorre los camiones buscando el primero donde quepa el paquete
- Si cabe, lo asigna y se llama a sí misma con índice + 1
- Si no cabe en ningún camión, se llama a sí misma con índice + 1 saltando el paquete

Cada llamada reduce el problema en 1 paquete, garantizando que eventualmente llega al caso base.

Análisis de complejidad

Complejidad temporal:

$$O(n * m)$$

1. La función se llama recursivamente una vez por cada paquete: profundidad n
2. En cada llamada se recorren máximo m camiones: $O(m)$
3. No hay backtrack ni exploración de múltiples ramas
4. Total: $O(n * m)$

Complejidad espacial:

$$O(n)$$

La recursión genera una pila de llamadas de profundidad máxima n , una llamada por cada paquete. No se almacenan combinaciones adicionales, solo la asignación actual.

Medición empírica de tiempos

Caso	Paquetes	Camiones	Tiempo real
Pequeño	80	15	0.000854 seg
Mediano	700	120	0.001466 seg
Grande	5000	900	No termina

Comparación de tiempos contra fuerza bruta

Algoritmo	Pequeño	Mediano	Grande
Fuerza bruta	No termina	No termina	No termina
Recursivo	0.000854 seg	0.001466 seg	No termina

El recursivo resuelve los tres casos en microsegundos mientras que fuerza bruta no termina en ninguno. Esto se explica por la diferencia de complejidad: $O((m+1)^n)$ vs $O(n * m)$.

Sin embargo esta velocidad tiene un costo, ya que el recursivo no garantiza la solución óptima. Toma la primera opción válida sin evaluar si existe una mejor distribución.

Análisis:

La diferencia en tiempos se explica directamente por la complejidad de cada algoritmo. La fuerza bruta tiene complejidad $O((m+1)^n)$, lo que la hace inviable para cualquier caso de prueba. El recursivo tiene complejidad $O(n * m)$, lo que le permite resolver el caso pequeño y mediano en microsegundos, pero falla en el caso grande por el límite de llamadas recursivas que Python permite.

Conclusión:

No existe un algoritmo perfecto, siempre hay un balance entre velocidad y optimalidad. La fuerza bruta garantiza la solución óptima pero no termina en ningún caso. El recursivo es mucho más rápido pero no garantiza optimalidad y tampoco escala al caso grande. Para los siguientes algoritmos (greedy, backtracking, divide y vencerás) se espera un mejor equilibrio entre velocidad, escalabilidad y calidad de solución.

Entrega 2 Proyecto final

Análisis y diseño de algoritmos

Índice

Solución Greedy	2
Idea general del algoritmo.....	2
Decisión local óptima.....	2
Análisis de complejidad — Greedy.....	3
Análisis crítico — Greedy	5
Conclusión — Greedy	6
Solución con Backtracking	7
Idea general del algoritmo.....	7
Poda implementada	7
Restricciones utilizadas para podar	8
Análisis de complejidad — Backtracking.....	8
Medición empírica de tiempos — Backtracking	9
Análisis	9
Conclusión — Backtracking.....	9
Comparativa parcial	10

Entrega 2 – Greedy y Backtracking

Solución Greedy

Idea general del algoritmo

La estrategia greedy utilizada en este proyecto busca resolver el problema de asignación de paquetes tomando decisiones rápidas y locales en cada paso del proceso.

El algoritmo recorre los paquetes en orden y, para cada uno, intenta asignarlo inmediatamente al primer camión disponible donde el peso no exceda la capacidad restante.

La idea principal es:

Maximizar rápidamente la cantidad de paquetes entregados sin explorar múltiples combinaciones posibles.

A diferencia de fuerza bruta o backtracking, greedy no analiza escenarios futuros ni reconsidera decisiones tomadas anteriormente. Una vez un paquete es asignado, esa decisión permanece fija durante toda la ejecución.

Decisión local óptima

La decisión local óptima implementada consiste en:

“Asignar el paquete actual al primer camión donde pueda ser transportado sin superar la capacidad máxima.”

Esto permite reducir drásticamente el tiempo de ejecución, ya que evita explorar todas las configuraciones posibles del problema.

Sin embargo, esta decisión se toma únicamente con la información disponible en el momento actual, sin evaluar cómo puede afectar futuras asignaciones.

Funcionamiento del algoritmo

El procedimiento general del algoritmo greedy es el siguiente:

1. Se inicializan las cargas actuales de todos los camiones.
2. Se recorren los paquetes uno por uno.
3. Para cada paquete:
 - Se revisan los camiones en orden.
 - Si el paquete cabe en un camión:
 - Se asigna inmediatamente.
 - Se actualiza la carga ocupada.
 - Si no cabe en ningún camión:
 - El paquete se descarta.
4. El proceso continúa hasta procesar todos los paquetes.

El algoritmo nunca retrocede ni reorganiza paquetes ya asignados.

Análisis de complejidad – Greedy

Complejidad temporal

Sea:

- n = cantidad de paquetes
- m = cantidad de camiones

El algoritmo primero ordena los paquetes por prioridad, lo que tiene un costo de $O(n \log n)$. Luego recorre los paquetes uno por uno y para cada uno revisa hasta m camiones.

Por lo tanto, la complejidad temporal es:

$O(n \log n + n * m)$ que se simplifica a **$O(n * m)$** para valores grandes de m .

Justificación paso a paso

1. Se ordenan los n paquetes por prioridad $O(n \log n)$
2. El algoritmo recorre los n paquetes
3. Para cada paquete puede revisar hasta m camiones
4. Cada verificación de capacidad es una operación constante
5. Total: $O(n \log n + n * m)$ - **$O(n * m)$**

Complejidad espacial

Se almacena un arreglo de cargas para los m camiones y la asignación de hasta n paquetes.

$O(n + m)$

Medición empírica de tiempos — Greedy

Caso	Paquetes	Camiones	Combinaciones	Tiempo real
Pequeño	80	15	$80 * 15 = 1.200$	0.000128 seg
Mediano	700	120	$120 * 700 = 84.700$	0.001667 seg
Grande	5000	900	$900 * 5000 = 4.500.000$	No termina

Análisis crítico – Greedy

La principal ventaja del enfoque greedy es su velocidad. El algoritmo toma decisiones inmediatas basadas en la prioridad de cada paquete, evitando explorar combinaciones complejas.

Sin embargo, esta rapidez tiene un costo importante: greedy no garantiza encontrar la solución óptima. El problema ocurre porque el algoritmo solo analiza el estado actual y no considera cómo una decisión puede afectar futuras asignaciones.

Contraejemplo

Supongamos:

Camiones

Camion	Capacidad
C1	10
C2	10

Paquetes

Paquete	Peso	Prioridad
P1	9	3
P2	9	2
P3	2	1
P4	2	1

Resultado usando Greedy

Greedy ordena por prioridad y asigna

P1 - C1,

P2 - C2.

Ambos camiones quedan con capacidad restante de 1,

P3 y P4 no caben.

Resultado: 2 paquetes entregados.

Solución óptima

Existe una distribución mejor:

- $P1 + P3 \rightarrow C1$
- $P2 + P4 \rightarrow C2$

Resultado final:

- 4 paquetes entregados.

Esto demuestra que la estrategia greedy puede desperdiciar capacidad útil y perder soluciones mejores.

Conclusión — Greedy

El algoritmo greedy representa una solución eficiente para el problema de asignación de paquetes.

Su decisión local óptima — asignar primero los paquetes de mayor prioridad — mejora la calidad de la solución respecto a una asignación sin criterio, pero no garantiza optimalidad.

Por esta razón, greedy es adecuado cuando:

- Se prioriza rapidez
- El tiempo de respuesta es crítico
- El tamaño de entrada hace inviables soluciones exhaustivas.

Solución con Backtracking

Idea general del algoritmo

La estrategia de backtracking busca explorar múltiples combinaciones posibles de asignación de paquetes a camiones.

A diferencia de greedy, este enfoque sí reconsidera decisiones y evalúa distintas configuraciones antes de elegir la mejor solución.

El objetivo principal es:

Maximizar la cantidad total de paquetes entregados respetando las restricciones de capacidad.

El algoritmo explora recursivamente diferentes posibilidades y conserva únicamente la mejor solución encontrada.

Funcionamiento del algoritmo

Para cada paquete existen varias decisiones posibles:

1. Asignarlo a un camión válido.
2. Probar otro camión.
3. O no asignarlo.

El algoritmo realiza:

- una decisión,
- explora recursivamente,
- y luego deshace la decisión para probar otras alternativas.

Este mecanismo permite recorrer el espacio de soluciones posibles de forma sistemática.

Poda implementada

Debido al enorme número de combinaciones posibles, el algoritmo incorpora técnicas de poda para reducir el espacio de búsqueda.

La poda principal utilizada consiste en verificar:

“¿Aún es posible superar la mejor solución encontrada?” Para

ello se calcula:

- la cantidad actual de paquetes asignados,
- y los paquetes restantes por procesar.

Si incluso asignando todos los paquetes restantes no es posible superar la mejor solución actual, la rama se elimina inmediatamente.

Restricciones utilizadas para podar

1. Restricción de capacidad

Un paquete solo puede asignarse si no supera la capacidad restante del camión. Esto evita generar soluciones inválidas.

2. Poda por cota superior

Si: *paquetes actuales* + *paquetes restantes* no supera la mejor solución encontrada la rama deja de explorarse. Esta poda reduce considerablemente la cantidad de combinaciones evaluadas.

Análisis de complejidad – Backtracking

Complejidad temporal (peor caso)

Sea:

- n = cantidad de paquetes
- m = cantidad de camiones

Cada paquete puede asignarse a cualquiera de los m camiones, o no asignarse. Por tanto, el número aproximado de combinaciones posibles es: $(m + 1)^n$. La complejidad temporal en el peor caso es: $O((m + 1)^n)$, Esto corresponde a crecimiento exponencial.

Complejidad espacial

El algoritmo almacena:

- Cargas actuales
- Asignaciones parciales
- Llamadas recursivas

La complejidad espacial aproximada es:

$$O(n * m)$$

Caso promedio con poda

Aunque teóricamente el algoritmo sigue siendo exponencial, en la práctica las podas reducen significativamente el número de ramas exploradas.

Esto produce mejoras importantes respecto a fuerza bruta.

Sin poda, el algoritmo sería prácticamente inviable para entradas medianas.

Medición empírica de tiempos – Backtracking

Caso	Paquetes	Camiones	Tiempo real
Pequeño	80	15	0.001530 seg
Mediano	700	120	0.3733164 seg
Grande	5000	900	No termina

Análisis

Backtracking obtiene mejores soluciones que greedy debido a que explora múltiples configuraciones posibles.

Sin embargo, esto incrementa considerablemente el costo computacional.

A medida que crece la cantidad de paquetes y camiones, el número de combinaciones aumenta exponencialmente.

Las podas permiten reducir parte de esta explosión combinatoria, pero no eliminan completamente el problema.

Conclusión – Backtracking

Backtracking ofrece una solución mucho más precisa que greedy y cercana a la optimalidad.

La incorporación de podas mejora significativamente el rendimiento práctico del algoritmo.

No obstante, debido al crecimiento exponencial del espacio de búsqueda, el algoritmo sigue siendo costoso para entradas grandes.

Por esta razón, backtracking resulta adecuado cuando:

- Se busca maximizar calidad de solución,
- El tamaño de entrada es moderado,
- Y se dispone de mayor tiempo de procesamiento.

Comparativa parcial

Algoritmo	Complejidad	Optimo?	Velocidad	Escalabilidad
Fuerza Bruta	Exponencial	Sí	Muy lenta	Muy baja
Recursivo	$O(n * m)$	No	Rápida	Media
Greedy	$O(n * m)$	No	Muy rápida	Alta
Backtracking	Exponencial	Sí	Media/Lenta	Baja

Discusión general

La diferencia principal entre los algoritmos radica en el equilibrio entre: velocidad, calidad de solución, y costo computacional.

Greedy prioriza rapidez sacrificando optimalidad.

Backtracking prioriza calidad de solución sacrificando rendimiento.

Fuerza bruta garantiza optimalidad, pero resulta inviable para entradas reales.

Recursivo es rápido y simple pero no garantiza optimalidad y falla en entradas grandes por el límite de llamadas recursivas.

Por ello, cada estrategia representa un trade-off diferente dependiendo de las necesidades del problema.

Entrega final Proyecto

Análisis y diseño de algoritmos

Índice

Definición del problema	2
Entradas	2
Salidas	3
Restricciones	3
Solución Fuerza bruta	4
Análisis de complejidad — Fuerza Bruta	4
Gráfica comparativa (tiempo vs tamaño de entrada).....	5
Análisis	5
Conclusión	5

Entrega 3 — Divide y Vencerás + Comparativa Final

Solución Divide y vencerás

Idea general del algoritmo

La estrategia de divide y vencerás aplicada a este problema consiste en dividir la lista de paquetes en subconjuntos más pequeños, resolver cada subconjunto de forma independiente y combinar los resultados en una única asignación final.

A diferencia de greedy o backtracking, este enfoque no evalúa el problema completo de una sola vez, sino que lo descompone hasta llegar a casos simples que pueden resolverse directamente.

Fases del problema

Fase 1 — Dividir

El algoritmo divide recursivamente la lista de paquetes en dos mitades iguales hasta llegar al caso base: una lista con un solo paquete.

Fase 2 — Vencer

Cuando la lista tiene un solo paquete, el algoritmo intenta asignarlo al primer camión donde quepa respetando la capacidad máxima. Si no cabe en ningún camión, el paquete se descarta.

Fase 3 — Combinar

Los resultados de ambas mitades se combinan automáticamente porque comparten las mismas estructuras de cargas y asignación. Cada mitad actualiza el estado de los camiones, y la siguiente mitad continúa desde ese estado.

Análisis de complejidad

Complejidad temporal

La relación de recurrencia del algoritmo es:

$$T(n) = 2T(n/2) + O(m)$$

Donde $O(m)$ representa el trabajo de asignar un paquete revisando hasta m camiones en el caso base.

Aplicando el Teorema Maestro con $a=2$, $b=2$ y $f(n)=O(m)$:

Como m es constante respecto a n , $f(n) = O(1)$ en términos de n , lo que da una complejidad de $O(n * m)$.

Complejidad espacial

El algoritmo genera una pila de llamadas recursivas de profundidad $\log(n)$, y en cada nivel almacena referencias a sublistas. La complejidad espacial es $O(n + \log n)$ que se simplifica a $O(n)$.

Medición empírica de tiempos

Caso	Paquetes	Camiones	Tiempo real
Pequeño	80	15	0.0000935 seg
Mediano	700	120	0.001338 seg
Grande	5000	900	0.05358 seg

Comparativa final

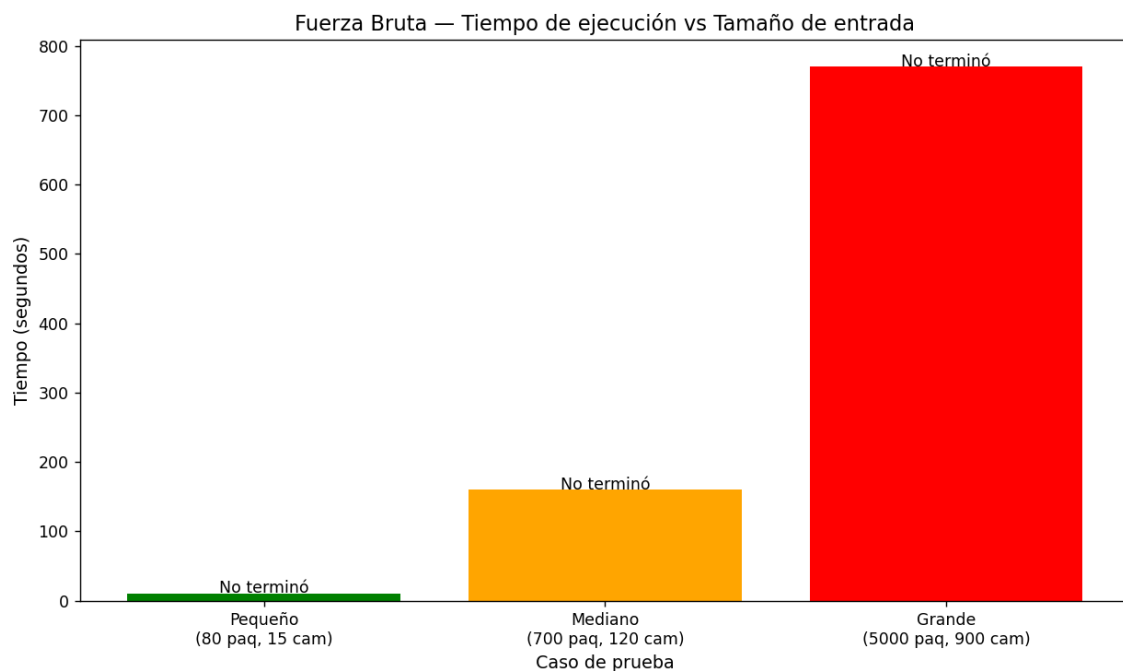
Tabla comparativa

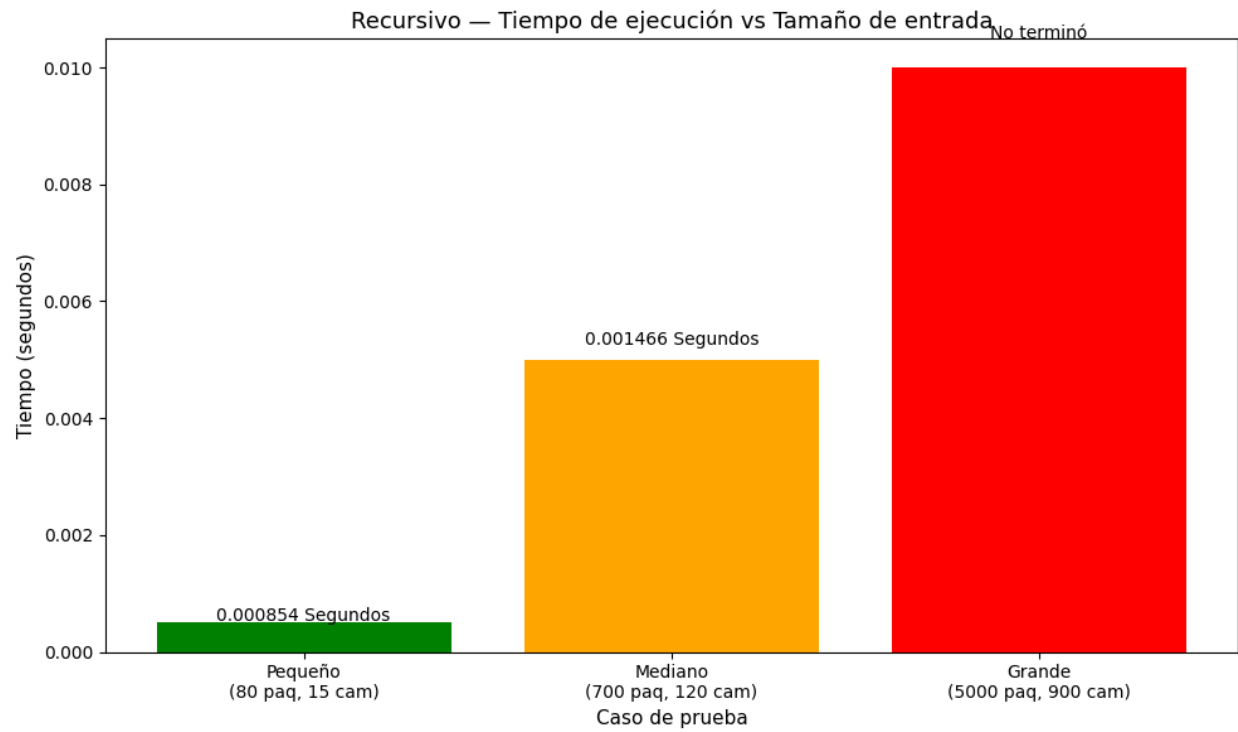
Técnica	Complejidad temporal	Complejidad espacial	¿Solución óptima?	Tiempo pequeño(80 paq, 15 cam)	Tiempo mediano(700 paq, 120 cam)	Tiempo grande(5000 paq, 900 cam)
Fuerza bruta	$O((m+1)^n)$	$O(n * m)$	Sí	No termina	No termina	No termina
Rekursiva	$O(n * m)$	$O(n)$	No	0.000854 seg	0.001466 seg	No termina
Greedy	$O(n * m)$	$O(n + m)$	No	0.000128 seg	0.001667 seg	No termina
Backtracking	$O((m+1)^n)$	$O(n * m)$	Sí	0.001530 seg	0.3733164 seg	No termina
Divide y Vencerás	$O(n * m)$	$O(n)$	No	0.0000935 seg	0.001338 seg	0.05358 seg

Gráficas de las soluciones

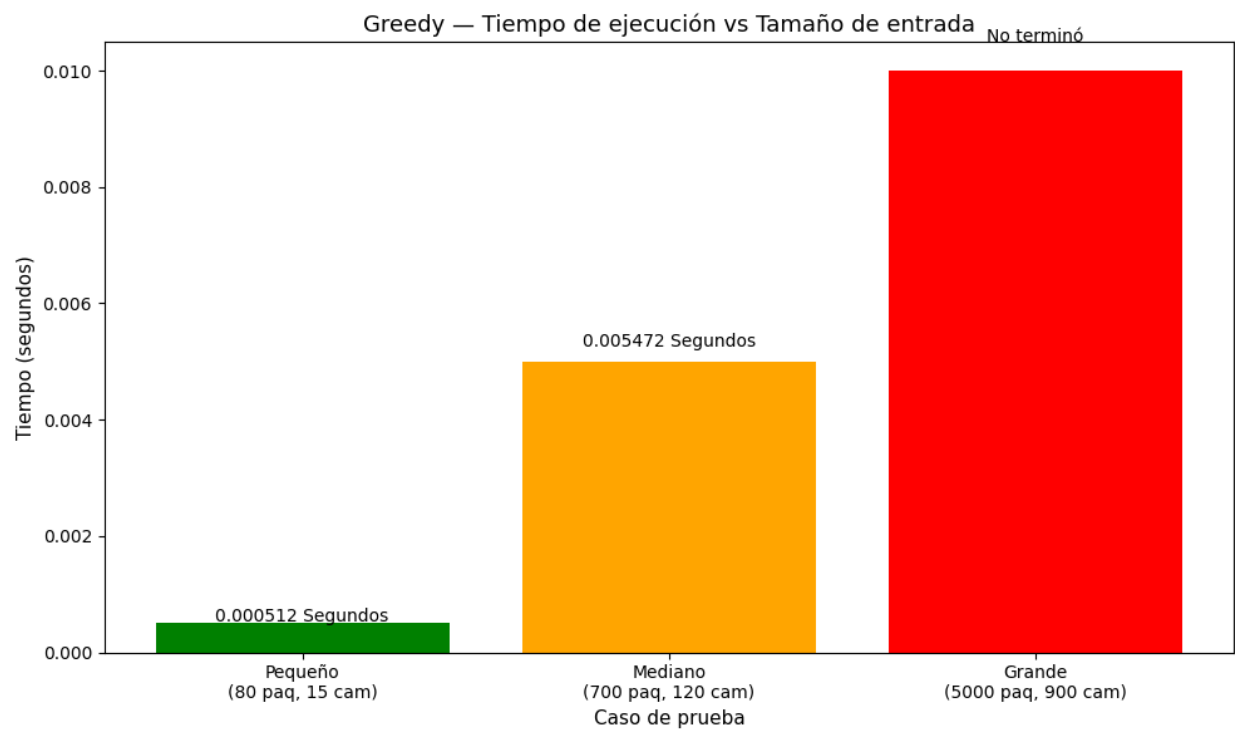
Fuerza bruta

Rekursiva

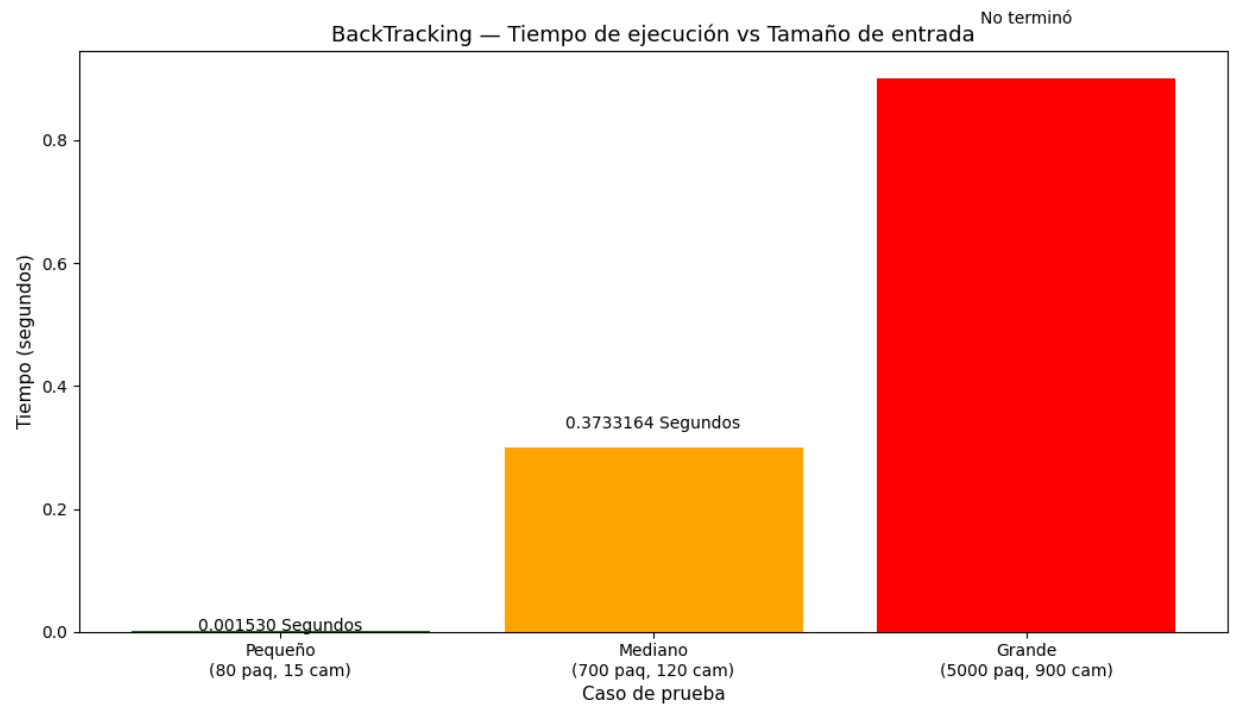




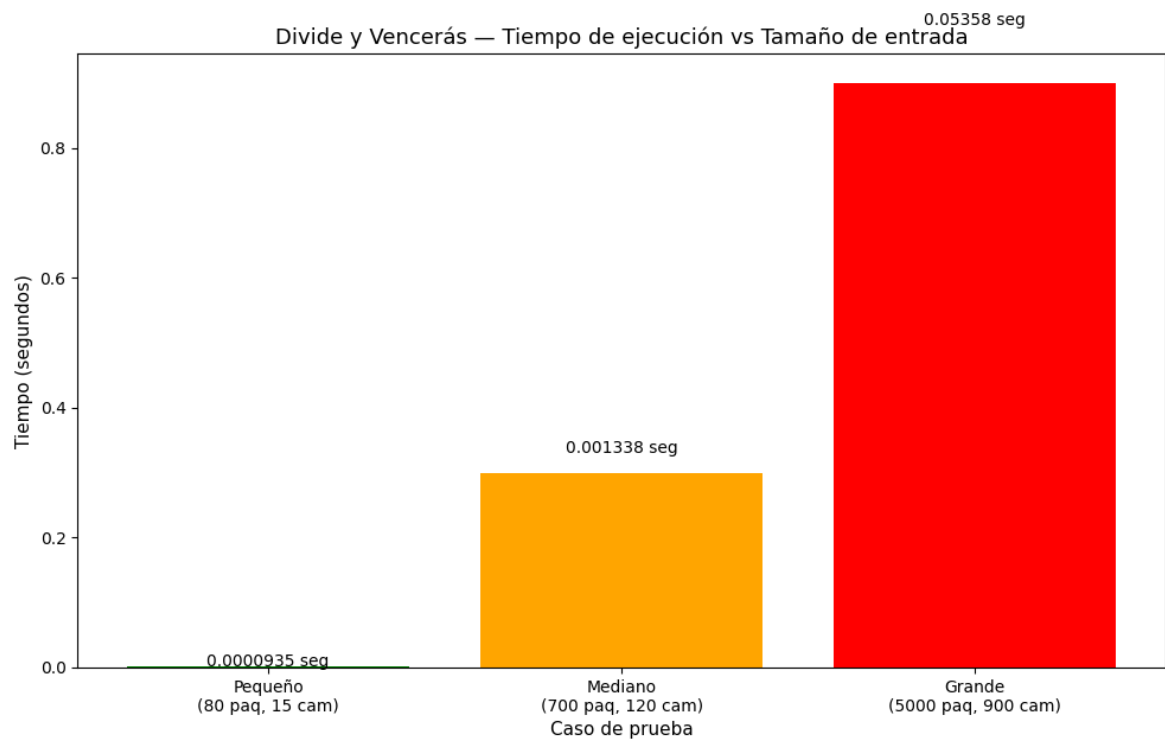
Greedy



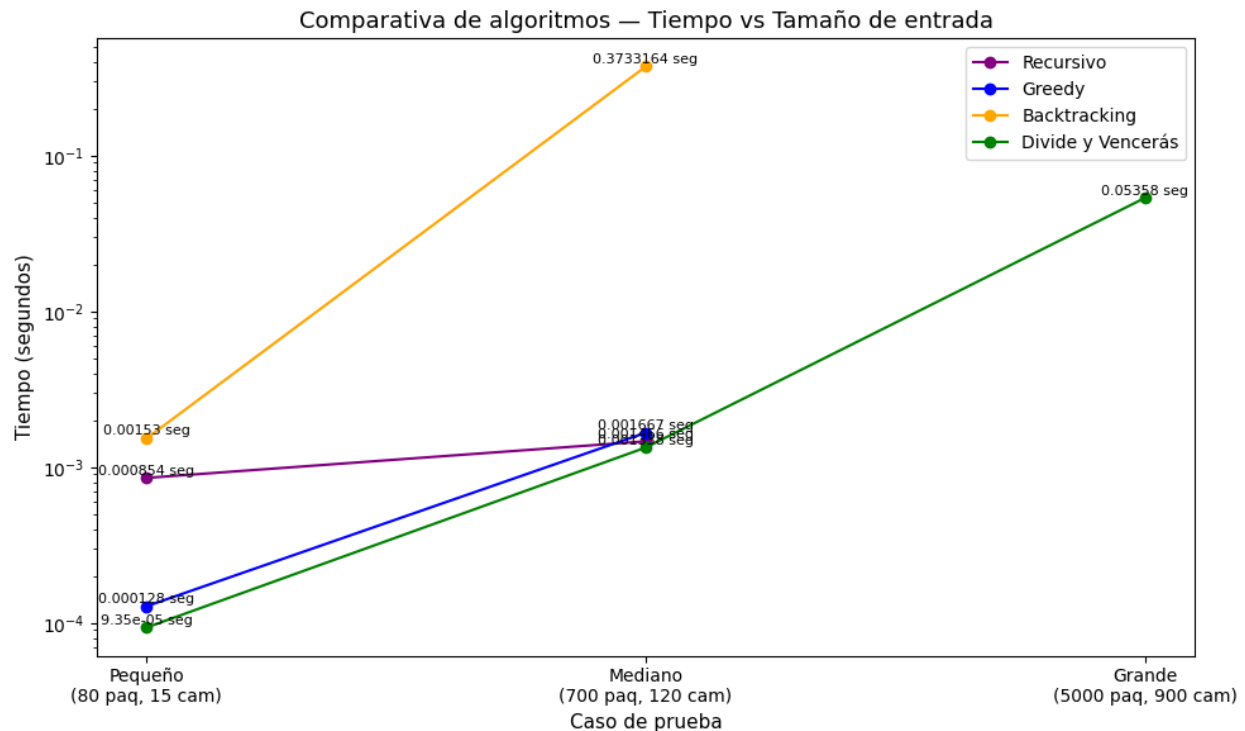
Backtracking



Divide y vencerás



General



Analisis

¿qué técnica es la más adecuada para este problema?

No existe una única técnica óptima para este problema, ya que la elección depende del contexto y las prioridades del sistema.

Pero podemos analizar los resultados obtenidos:

Divide y Vencerás es la técnica más adecuada para sistemas reales porque es el único algoritmo que logró resolver los tres casos incluyendo el grande (5000 paquetes, 900 camiones) en un tiempo razonable de 0.05358 segundos. Escala bien a medida que crece la entrada y su complejidad $O(n * m)$ es predecible y manejable.

Greedy es la segunda mejor opción cuando el tamaño de entrada es moderado. Su complejidad es igual a divide y vencerás pero no pudo resolver el caso grande, ya que las llamadas recursivas acumuladas consumen más memoria y tiempo de procesamiento del esperado, limitando su rendimiento en entradas muy grandes.

Backtracking es adecuado cuando la calidad de solución es crítica y el tamaño de entrada es pequeño o mediano. Garantiza la solución óptima pero su crecimiento exponencial lo hace inviable para entradas grandes.

Recursivo es útil cuando se necesita una solución rápida y válida sin importar la optimalidad. Es el más simple de implementar, pero falla en entradas grandes por el límite de llamadas recursivas.

Fuerza bruta queda descartada para cualquier uso práctico ya que no pudo resolver ninguno de los tres casos. Su único valor es académico, en este contexto podemos tomarlo como referencia para verificar que los demás algoritmos producen soluciones correctas.

A tener en cuenta

Cada algoritmo puede devolver una solución diferente porque tienen estrategias distintas:

Fuerza bruta: solución óptima (la mejor posible)

Recursivo: primera opción válida encontrada, sin criterio de selección

Greedy: buena solución basada en prioridad, pero no garantiza el óptimo

Backtracking: solución óptima igual que fuerza bruta, pero más rápido gracias a la poda

Divide y vencerás: depende de cómo se divida, puede variar.

Conclusiones

El desarrollo de este proyecto permitió analizar y comparar diferentes técnicas algorítmicas para resolver el problema de asignación de paquetes a camiones, evidenciando cómo la complejidad de un algoritmo influye directamente en su rendimiento y viabilidad práctica. A través de las pruebas realizadas, se observó que los algoritmos de complejidad exponencial, como fuerza bruta y back tracking, aunque garantizan soluciones óptimas, presentan tiempos de ejecución demasiado altos para manejar entradas grandes, lo que limita considerablemente su uso en escenarios reales.

Por otro lado, las técnicas recursivas, greedy y divide y vencerás demostraron un mejor desempeño en términos de eficiencia y escalabilidad. En especial, divide y vencerás fue la solución más estable y eficiente, ya que logró resolver todos los casos de prueba, incluyendo el caso grande de 5000 paquetes y 900 camiones, manteniendo tiempos de ejecución bajos y consistentes. Esto demuestra que los algoritmos con complejidad $O(n^*)$

m)) son mucho más adecuados para aplicaciones que requieren procesar grandes volúmenes de información en tiempos razonables.

Finalmente, este proyecto permitió comprender que no existe una única técnica ideal para todos los problemas, sino que la elección del algoritmo depende de las necesidades específicas del sistema. Si se busca la solución óptima, algoritmos como backtracking pueden ser útiles en entradas pequeñas; mientras que, si se prioriza velocidad y capacidad de escalamiento, técnicas como divide y vencerás o greedy resultan más apropiadas para entornos reales.